

00:00

Yeah, good morning everyone and thanks for joining me for week seven, a topic which is focusing on data analytics. The main topic of today is melting, that's probably not new because we briefly referred to that in previous weeks, but I'm going to expand it a bit more. And the opposite of melting, which is pivoting. Then you get familiar with pivot tables, something very practical, which is binning, we are going to talk about today. Then there are two small techniques, which are enlarged values and ranking in a data frame. So today's content is not heavy. And I hope we get some time because I prepared some quiz for you. We can practice in the middle of today's session if you're interested.

01:01

So the first part was melting. And I'm going to tell you how you can melt a lot of data, which means after melting, we will have long-form data. So if you want to have some idea what melting really means, I got a message. I hope everything's fine. Guys, you're doing well. Can you hear me? Yeah, we got a message. I hope everything's fine. Guys, you're doing well? Can you hear me? Yeah, we can. Perfect. So, melting data in panels, guys. If you compare a data frame that I have on the left-hand side, okay, this is called a wide data frame because each column shows a category. By category, we mean group, class, whatever you want. Two separate things, two different labels.

01:54

So here, as you see, I have a column student that shows the name of a student, then two separate subjects or units. This is showing me a wide data frame because I have different categories in different columns. And what you see on the right is the same data frame after melting. On the right, I have melted my data. And as you see, this is a longer data frame. And by longer, I mean I have more rows. I have four rows here. In the previous one, I have only two rows. This is called a long frame. What about now? Can you hear me now? Yeah, we can. Okay, cool. I don't know what's wrong with Teams. It happens very regularly. So, guys, let's continue.

05:18

Here I was going to tell you how you can melt a data frame. That's the procedure, guys.

ys. This is my wide table, and this is the melted one. But how does it work? We have a function called melt. It's built into Pandas. That's why you will see `pd.melt`, which means melt is built inside Pandas. So when you call `pd.melt`, you open parentheses and define what you want to do. The first step is calling the data frame, which is wide, and you're going to work on that to melt it. There are some options or arguments or parameters that you have defined well. What are the parameters? The first one is `id_vars`, or this stands for variables. You should specify which column of the existing data frame you want to have as an identifier. So here, as you see, I say the student column wants to be an ID or identifier. So look at your wide table. You have a column named student. You use it as your identifier. This always comes as your first column.

06:40

And what happens? You can see whatever you have in your wide table is repeated here. It's always repeated. You will see why, because you choose some categorical columns from your wide table, as you see. You name them math and science. We choose these columns from our wide table to form a new column for me. And as you see, I named these columns subjects. So I have a new column which consists of my previous columns. Math and Science. Then I have values for this column. These values are coming from a score. I named it score, and they are coming from Math and Science as you see. So what happened? I keep one of my columns as an identifier, which is fixed: students. This column gets repeated. Why? Because I'm repeating my columns, math and science. This new column gets a new name. I simply gave it a new name, subject, and another column which shows the values that I have. So nothing new is added. I have the same data, but in different formats.

08:17

Then there are some examples here. I may go to the code I prefer. Go to the code, guys. Let me get this and probably close this one and have your data frame. Yep, that's better. Okay guys, you see after importing your libraries as we always do, let me increase the font. If it's not visible, let me know. Okay, I think that's the largest font I can have on my screen. You see, guys, here I have a data frame, which is called cars. Simply, I use the shape, and I get the shape of my data frame because I'm going to see how many rows and columns I have. Shape gives you an output, which is a tuple. Tuple means two numbers inside parentheses. The first number, 205, shows the number of rows, and 5 is the number of columns. And this is the first couple of rows of my data

frame, which is named cars. So as you see, I have 5 columns and 205 rows. I simply picked the first three ones here. So I get some idea what I have inside, but I'm going to melt it now.

09:47

These are explanations about each parameter that I'm going to use. As you see, cars_melted has been created by calling the melt function in Pandas. I used the data frame that I originally had. I'm showing price as my identifier. So guys, this column price is going to be my identifier. So it becomes my first column. Then I'm going to pick these two columns, engine size and curb weight, as those columns that I'm going to use as a new column. And I name this column feature. Then I have another column. I call it feature_value. So what you technically do is pick some columns that you want to be used, put together, stack together as rows to create a new column for you. The name of this new column is feature. That's arbitrary. I simply gave it a name. And another column you create, you give it a name. I simply called it feature value.

11:16

So what Python does is restructure your existing data frame. It doesn't do any operation. It simply restructures. So let's see what happens if I do that. Initially, I had the cars data frame. I printed the shape. It had 205 rows. But as you see, after I melt it and get the shape of the melted data frame, the number of rows becomes 410, which is exactly twice my original data frame. Because if you remember, I had these two columns and repeated them. I know that each column by itself has 205 elements. So then I put them back together, it doubles, right? If you want to get a very clear understanding, you should print a very long view of the shape of your data frame. Here I don't have enough space. That's why I simply print the first three rows. You can print the first 50 rows and see how it shows itself.

12:29

I printed the first couple of rows of my melted data frame. As you see, the new column feature has engine size and curb weight. If you print longer, you will see curb weight also appears. Then there are some values. These values are taken from my original data frame. Here I played a trick. I'm just going to show you some part in the middle of my data frame to give you some idea what I mean by melting. I simply got the length—how many rows I have in my data frame, which is melted—and divided by tw

o, which means I'm looking at the middle of my data frame. When you look at this part of your data frame, you see that in the feature column, you have engine size and curb weight. These columns are exactly the two columns that I wanted to be stacked together. My original columns, engine size and curb weight, are stacked together here. That's why the number of rows becomes doubled. And the values are taken from your data frame and put there for you. The thing you should be aware of is arranging the columns that you want.

14:14

So, as you see, you have a very long table because the number of rows became double. If you had three columns in here, the number of rows would be triple, because everything gets repeated for each of these columns. That's why the index repeats as well. If you go to my simple example, you will see that Ali and Sarah keep repeating. Okay, that's how you melt a data frame. It's about restructuring the data frame. Let's see how you can plot this, guys. I'm going to plot the melted data frame using `sns.relplot`, or relational plot. We worked with them in week four or five. Please watch those videos. These relational plots can create meaningful and informative visualizations. Here I'm telling relational plots to use the X-axis for feature value and the Y-axis for price. I have two numerical variables on the X and Y axes: price versus feature value.

16:06

There is something very cool, an option in relational plots, which is `hue`. `Hue` looks at a column that has a categorical color variable. It automatically detects things. If, for example, a feature value or price has engine size, it gives a specific color to that. If another row has curb weight, it gives a separate color so it can be identified. So you get a beautiful scatter plot. By default, relational plots give you a scatter plot. If you want a line, you can simply change the plot. I want you to try it yourself. I wrote everything for you. You can uncomment it by pressing `Control+Slash` and run it again. Then instead of dots, you will have lines. A scatterplot is technically more meaningful when it comes to comparing relationships. At first glance, you can see that the price is more or less the same, but the weight is much greater than engine size. So it may be a better idea to plot them separately.

17:40

As you see in the first plot on the left, you can see engine size. That's the title of my f

figure. In the other one you have curb weight. The beautiful thing is that if you plot them separately, you have better visualization because the X-axis for engine size starts from 1,500 and the other starts from 50. In the original figure, you had a huge gap because most engine-size values are compacted in a close range toward zero, while curb weight has much larger numbers. If you plot them separately, you can see how it spreads better and get a clear understanding. How did I do that? I used relational plots and asked them to have X values separate from each other. In the first X-axis I start from 50, and the other starts from 1,500. It automatically looks at the data and draws it meaningfully.

19:12

Why did I have two separate subplots? By using `col`, if you remember, I can tell my relational plot to put my data in separate columns or separate rows. You can use `row` if you want your subplots to appear in separate rows. Here I have columns, so each subplot is in a separate column. It automatically goes to the feature column of my data. My data is `cars_melted`, and it looks at the feature column. After going through this column, it recognizes that I have engine size and curb weight as two separate things, so it plots them separately for me. Beautifully and automatically, because I don't want to have a shared X-axis. The `facet` keyword tells it that I don't want the X-axis to be shared. It looks at the values for each category and arranges them accordingly. I don't have any value before 50, so it ignores that range. I don't have anything above 400, so it ignores that automatically and I get a beautiful distribution.

21:07

If I wanted everything before 1,500, I couldn't get something beautiful. It would look like this, which is not clear. It looks like my blue data points are linearly distributed, but when I look carefully, I see the distribution. That's important, guys. The way that you specify the interval and range can tell you different stories. It's spread meaningfully now, but here the points are congested. So be careful about all these options that your SNS has and draw meaningful visualizations. Now I'm going to talk about categorization using aggregation functions. Aggregation means you get many numbers and out of those numbers get one single number. For instance, summation: you give a large data set and want to sum the values in one column. Out of all the numbers, it gives me one number. Or finding the maximum or mean. All these are aggregation methods or functions.

02:14

After reading my pickled data frame, I draw the first couple of lines. We worked with that data frame last week as well. This is my index starting from 16. We usually do not want things messy. We prefer to start from zero. That's why we use `reset_index`. It resets my index, drops the previous index, and gives me an index starting cleanly from zero. This will be my fires data frame from now on. As you see, I have different types of columns: object or text in the first column, integer in the second, categories or labels in another, the next two columns are date-time, and the rest are numerical data. We're going to do some aggregation on this data frame. Most methods I'm going to use today are repetitive, so I won't go into detail because I prefer you to go through the quiz. If you have any issue, drop me a message in the chat and we will discuss it after class.

02:23

I'm calling `groupby` on my fires data frame based on state. It means group everything according to states. By default, when I apply `groupby`, that column becomes my index. The state appears in bold as the index, and the other columns remain. I want to calculate the mean or average of these columns, and I specify `numeric_only`. You may try it at your end. If you remove this option, you may get an error depending on the version of Pandas or Python installed. If I want to calculate any operation—sum, mean, max, whatever—and I have columns that are not numeric, Python may raise an error. My data frame has different data types, like date, time, categorical, or object variables. So you should clearly tell Python to perform the operation only on numeric columns. It calculates the sum only for the numeric columns and ignores the others.

25:51

Writing `numeric_only` is good, but sometimes it is not enough. Fire year is also a numeric value, but calculating the average year does not actually give a good meaning. Sometimes it is better to clearly tell Python which columns I want to apply the operation to. After grouping, you can select the columns on which you want to perform the operation. Before doing this, be aware that when you `groupby` and specify a column or list of columns, there is another option, `sort`, whose default value is `true`. This means it sorts things automatically. If you look at state after `groupby`, you will see it is in alphabetical order. In your original data frame the states were mixed and had no order.

er, but groupby sorts them automatically. If you do not want to change the order, use `sort=False`. Then the states appear in exactly the order in which they appeared in the original data frame. Sorting starts from the smaller alphabet, number, or date, so it goes from earliest to latest.

28:55

There are two different ways of working on the data. You can groupby easily, and when applying an aggregation function, tell Python to work only on numeric columns. But if you want to be more specific, you can select the columns. These approaches give the same output, but the second is more specific. It avoids calculating something meaningless, such as the mean of a year. Working as a data scientist is not only about understanding numbers; you should also be aware of how to tell a story with your data. If I calculate the mean of the years in which bushfires happened, it probably is not meaningful. You should have domain knowledge about what is meaningful and what is not. The columns used for groupby appear as the index: state, fire year, and fire month. I categorize each state, then each year in that state, and then each month of that year. It goes through all months, moves to the next year, and then to the next state. It is like a loop repeating until the whole data has been processed.

31:16

I recommend printing a very long list of your data frame so you can see how things are repeated. Now I'm going to do more with groupby. I printed our data frame again. Everything is clean. I'm going to group it according to fire year. Fire year becomes my new index by default. If you don't want it to be indexed, you can set an option. I wanted to calculate the sum, and it applies only to the numeric columns, which are the last three columns. It calculates the sum for me. You can do many operations on top of each other; we call it chaining. Everything can happen in one line, but sometimes it is clearer to split operations into different lines, give each operation a separate name, and debug the code more easily. If you use `as_index=False`, the grouping column remains a column instead of becoming the index. The default is true, which is why whatever you group by appears as the index.

34:56

We can do several aggregation functions at the same time on one or several columns. I use groupby for three columns but keep them as columns by setting `as_index=False`.

Then I use `agg`, which stands for aggregation, and give a list of the operations I want to perform. These operations must be aggregation functions; they cannot be arbitrary functions. Aggregation methods take a group of numbers and give one number. Here it calculates sum, count, and mean for acres burned and days burning. Because `s_index` is false, the columns used for `groupby` remain columns and are repeated. For each state, I have all years, and for each year I have all months. This is like a loop and multilevel indexing. The next version is more or less the same, except `groupby` is used as the index. The numeric values are the same. You can perform different operations on different columns at the same time: one operation one column, one operation on several columns, several operations one column, or several operations on several columns.

38:29

This is the most versatile version. You can do different operations on different columns. I group by three columns and keep them indexed. Inside the aggregation structure is a dictionary: curly brackets containing key-value pairs. Each pair has a string, a colon, and a value, and pairs are separated by commas. Wherever you see the key, you apply the listed aggregation functions to that column. For acres burned, it calculates sum, max, and mean. For days burning, it calculates sum and mean. For fire name, it gives the count. It is possible to do different operations on various columns. So it was all about melting data. Melting, or having long format, is preferable for computers because they can perform operations more easily. Wide tables or pivot tables are more understandable for humans. The wide format is easier for us to read, while the long format can look messy.

41:03

The way to create a wide format is called the pivot method. Pivot spreads data into different columns. Instead of many rows, you have more columns, and those columns show unique values. Pivoting is the opposite of melting: melt forms a wide table into a long table, while pivot converts a long table into a wide table. Both reshape or restructure the data; they do not change the data. An important point about pivoting is that you should have unique values in your table. If values are repeated, you may have issues. For now, remember what pivoting does. I printed the data set I was going to work on, called `fires`, and the first two rows. Some things are repeated—year and state—and that can be confusing for humans, especially when grouping by multiple le

vels, although it is easier for computers.

43:13

I made a list of states. Each categorical variable or text value stands for one of the states, such as Indiana and Texas. I group by the two categorical columns, state and fire year. I do not want them to be indexed; I want to keep them as columns, so I set `as_index=False`. These columns remain columns and keep repeating. State is AK for all the years, and year 1992 keeps repeating until all the years have finished. Then it goes to the next state and all years are shown again. I calculate the sum for numeric columns only. The data frame has three numeric columns, and it calculates their sums. I want to filter the data more specifically using the query method. Query picks rows based on conditions defined for your columns. I want only rows in which the state value is in the list I defined. The `at-sign` means checking values in a list. The list contains AK, CA, ID, and TX. Printing only ten rows does not make this clear, so print a longer data frame to see the result.

46:19

Now I'm going to do pivoting. So far I simply printed part of the data and filtered it; I did not apply pivoting. For pivot, I define index, the columns that I want to become new row values. I pick fire year to become my index. If you compare fire year in the original data frame, it becomes the index. I set columns equal to state. The unique values in the state column become new column headers or labels. Python goes to the state column, finds the unique values—A, K, C, A, I, D, and X—and uses them as the new column labels. I had about 50 states in the US but did not want 50 columns, so I selected only a few states. When I pivot, only those unique values become the new headers. Then I tell Python to use acres burned for the values. The resulting data frame tells me, for example, that in 1992 I have specific acres-burned values for each selected state. This is simply restructuring; no operation is performed.

49:42

When it comes to plotting, nothing changed in the data. I apply the plot function. The visualization shows that over time, from 1995 to 2015, different states have different areas burned. This is not a very good visualization, so we try to calculate the mean. The mean decreases noise: jumps become less influential and outliers have less impact. But pivot and melt only restructure the data and do not calculate anything. For c

alculation, use the `pivot_table` method. Pivot tables can also handle repeated values, whereas `pivot` can raise an error when values are duplicated. `Pivot_table` has an aggregation function argument, `aggfunc`, whose default is `mean`. You can provide any aggregation function or a list of functions. It can both reshape data and calculate values.

52:47

I specified a few states because there are 50 and I wanted a smaller, cleaner data frame. Using `query`, I filtered rows that exist in those states, so only four states were selected. I named this data frame `Fires Top 4`. I want to improve the earlier messy visualization. Usually we calculate the mean to remove outliers, jumps, and noise. I choose `fire year` as the index because I want to see things over time. For each state, I get one column label. The values are acres burned. You can choose any numeric column to get different results and visualizations. The difference between `pivot_table` and `pivot` is the aggregation-function argument. `Pivot_table` can apply a function; `pivot` cannot. I use `mean` as the aggregation function. When I plot the result, the visualization is better, with less noise and clearer insights. It is wiser to use `pivot_table` for visualization because you can apply an aggregation function.

55:48

So, guys, it's one hour, and I have some quiz for you that would be fun. I was thinking I would show you this and give you 30 seconds for each question. Then you write the answer for me. I'm interested to see whether you got the concept clearly. Don't worry about remembering names. I'm not going to assess anyone by the quiz. I appreciate that you are here because some students did not come. It means a lot that you are attending the lecture, so don't worry about the results. Simply type what you think and drop your answer in the chat. Many people said B. What does `melt` do? It converts a wide table into a long table. Yes, that's correct. Thank you, guys. Let's look at the next question about finding the identifier and what happens to the columns.

57:51

Question two: many people selected B. You detected B, but remember that when we `melt`, columns get repeated, so the index should also get repeated. B is correct, but C is more correct because it also gets repeated. The next question is about counting columns. That might be tricky. If you have four columns to `melt`, you need to identify th

e identifier columns and the columns to be melted. How many columns does the melted table normally have? Don't worry if you do not remember the names; just use the concept. We have two identifiers, so the output definitely has two identifier columns. Then four columns are to be melted, but all four form one single column because melting takes several unique columns and puts them into rows with one name. The other column contains the values. So the output has the identifier columns, one new column for the former column names, and one column for the values.

01:01:42

The next question is about pivoting: unit values from subject build a wide table. That is correct. What happens if a duplicate occurs when using pivot? Pivot produces an error. That is why we use `pivot_table`, which can handle duplicated values. The main takeaway is that melting converts a wide table into a long table, while pivoting converts a long table into a wide table. The data should be unique for pivot. If it is not unique, you should use `pivot_table`. Now let's talk about binning. You have worked with categorical variables and different charts. By binning, I mean groups or categories of data. Binning puts continuous data into categorical ranges. For example, if you have a group of numbers, you can allocate categories such as small, medium, and large. The `qcut` function in Python does this for you. It converts continuous numbers into categories.

01:06:00

Binning helps you understand data better. University marks might range from 0 to 100. Instead of focusing on whether you got 81 or 82, which are very close values, you can put them into wider groups. You could label 80 to 90 as distinction and 90 to 100 as high distinction. The important point is giving a meaningful label to a range of numbers. Another benefit is easier comparison. If you have millions of numbers, you cannot inspect them all at one glance. Categorizing them into bins lets you see the bigger picture and makes visualization clearer. Binning also decreases noise: small variations may not matter, so you can ignore tiny differences and get a more general view of the data.

01:08:26

Pandas has functions that take care of binning. I went to my fires data frame and used `query` to filter rows. The query contains a string with the filtering condition: fire year

equals 2010 and days burning is positive. Then I drop rows with missing values and print the first three rows. `pd.cut` performs binning. It categorizes the data into four bins because I set bins to four. It selects the acres burned column and looks at the whole column, dividing its values into four groups. The output shows four categories and the intervals belonging to each bin. The first interval starts slightly below the minimum value, and the following bins allocate values that fall within their respective ranges.

01:12:00

The first number you see is an index label from the original data. An index means row labels, while the other labels are column labels. The interval shown in parentheses and square brackets tells you which values are excluded or included. A round bracket means excluded and a square bracket means included. The row specified by the index falls in that range, and the next row may fall in the same bin. Python calculates the bins by finding the range, which is maximum minus minimum. If the data runs from 20 to 100, the range is 80. That range is divided into four equal intervals. The number of boundary points is one more than the number of bins. Four bins require five points: a start point, an end point, and three points between them. Python performs the calculation automatically.

01:16:21

The negative lower boundary appears because of how Python calculates the bins. The area burned cannot actually be negative, but Python slightly moves the lower boundary below the minimum so that the smallest value is included. It considers a small percentage of the whole range and subtracts it from the minimum. If the range is large, this can produce a negative lower boundary. That is why the negative number appears. The data may also be imbalanced, meaning many observations are concentrated in one bin. Most natural data is imbalanced. Instead of allowing Python to calculate strange boundaries, you can define your own bins. If you want four bins, specify five boundary numbers. You can also give labels to the bins, such as small, medium, and large. This makes the result easier to understand.

01:20:57

`Value_counts` gives the number of each unique value. Most observations fall into a small bin, and some bins may contain nothing. This means the data is highly imbalanced.

ed or skewed. We can use `pd.cut` to specify boundaries manually, but sometimes it is difficult to choose them. `qcut` is another option. It creates approximately equal numbers of observations in each category rather than equal-width intervals. If the number of observations is not divisible by four, the groups will be approximately equal. Since the data is concentrated toward smaller numbers, `qcut` creates more bins in that area, effectively zooming into the data. The same observations and labels are retained, but `qcut` focuses more closely where the data is concentrated. Value counts then shows similar numbers in each category instead of the previous highly uneven distribution.

01:27:45

Now you can plot the result. I use `catplot`, which works with categorical variables and is a figure-level function. The visualization shows different counts of fires over time. Very large fires usually happen in months seven and eight, which are summer months in the Northern Hemisphere. That gives meaningful information from the data. Another simple technique is getting the `n` largest values. If you are interested in the top values in a column, `nlargest` returns an arbitrary number of them. Here I look at the engine size column, and the output shows the six largest values. The observations are sorted based on engine size, with the highest value first. You can apply this to several columns. When several columns are used, order matters: Python first considers the first column, and if values are equal, it uses the next column to break the tie.

01:31:21

Percentage change shows the change between successive values. Python compares the current value with the previous value, calculates the difference, and divides it by the previous value. The first value is always missing because there is no previous value. For example, from 100 to 120, the difference is 20, divided by 100, which is 20 percent and positive. From 120 to 90, the difference is negative 30, divided by 120, which is negative 25 percent. Positive means increasing and negative means decreasing. The last concept is ranking. Ranking is about position. By default, Python ranks in ascending order, so the smallest value gets rank one. In real life, rank one usually means the best or highest value, so you set `ascending=False`. Then the largest value comes first.

01:35:07

I categorized the data by state, which appears as the index, selected columns, calculated their sums, and created a new column called `acres_rank`. I selected acres burned and ranked it with `ascending=False`, so the largest burned area gets rank one. When values are equal, Python's default method assigns them the average rank. If two values occupy ranks two and three, both receive 2.5. You can use another method, such as `max`, to assign the maximum rank to tied values. Then both observations receive rank three. That is all for today. We have some time, so let's go to the quiz. In the quiz, remember that by default Python starts from the smallest value. For dates, it starts with the earliest date; for text, alphabetical order. If `ascending=False`, the order is descending, so the largest value comes first. That is why the answer is B.

01:42:10

`N-largest` means the five largest values when `n` equals five. If you follow the default order of parameters, you do not have to write their names. If you change the order, you should specify the parameter, such as `n=5`. The answer is C. That is all for today. Let me close this and go back to the code. I already uploaded the materials for week seven. We started with the `melt` function, which converts a wide table into a long table that is easier for a computer to understand. A wide table has each column representing a unique category. You specify IDs, and those IDs are repeated for each column that is melted.

01:45:30

Guys, if you want to leave, feel free. I'm just going to stay here until one and review the material. Thank you for joining me. Hopefully, see you next week. Here I chose two columns to form new rows, so the number of rows in the original data frame is doubled. If I had four columns, all four would be repeated, and the number of rows would be multiplied by four. Inspect your data frame: print its shape, count the rows and columns, and understand the data better. When you melt a table, the number of rows increases because it becomes long. When you pivot it or make it wider, the number of columns increases. Be careful about this. Also remember that `isin` checks whether values exist in a list. It means either one value or another. Boolean filtering first creates a series of true and false values, then the second operation keeps only the observations that are true and removes the false ones.

01:49:03

Relational plots and cat plots are figure-level functions, which means you can create several plots from the same data. The starting point does not have to be zero; the plot can spread the data into the space where it is needed and give a more realistic visualization. It is very important how you visualize your data. The same goes for binning. If binning is not done correctly, it can give you the wrong impression. As a data scientist, try different bin sizes, numbers of bins, and ranges instead of relying on the first result. We also talked about aggregation: `agg` can calculate max, mean, average, sum, and other functions. `numeric_only` lets you apply aggregation to numeric columns. Groupby columns appear as the index unless you set `index=False`. Pivot makes the table wider, although it can become more complicated. Calculating the mean can make visualization smoother by reducing noise.

01:53:34

We talked about binning continuous data into categories. With skewed or imbalanced data, most observations may appear in one bin while the others contain almost nothing. `qcut` helps by placing approximately equal numbers of data points into different categories, producing a nicer distribution and allowing you to zoom in where the data is concentrated. `nlargest` selects the rows with the highest values in a specified column, and with multiple columns the first column has priority while later columns break ties. Percentage change compares each value with the previous one; the first value is missing, positive means increasing, and negative means decreasing. Ranking defaults to the smallest value as rank one, but in real-world problems we often want the highest value to be rank one, so we use `ascending=False`. Always be the best friend of your data set. Use domain knowledge to understand what the numbers mean. If an area cannot be negative, adjust your bins accordingly. So let me stop recording, guys. Then we can talk about other things that you're interested in.